Velo x Noir Agentic Payment Hackathon — Builder Handbook

Updated Nov 5, 2026

Welcome

Build an AI agent that can pay, get paid and settle on Stellar, using Velo and the Noir SDK. The top 10 teams pitch face-to-face on Sunday, Nov 8, for a 10K prize pool.

Place	Prize
1st	5,000
2nd	3,000
3rd	2,000

Noir and Velo are two separate projects: Noir doesn't use Velo, and Velo doesn't use Noir. Your project is where they meet, and it must use both.

- **Noir** is a contactless payment wallet on Stellar, powered by the x402 protocol. Its Soroban contracts and SDK let an owner give an agent a key that can spend only within set limits.
- **Velo** is an application operations layer for Stellar, with checkout sessions, confirmed payments and signed webhooks.

The rule is strict: your agent's payments must go through the deployed Noir Testnet contracts, called with the Noir SDK, and you use Velo to build the other parts of your application.
Not registered through the Google Form yet? Register at forms.gle/wL8tPiCwGkK8YENV9. Without it, your project is not eligible for judging or prizes.
Join the Discord for updates and help during the hackathon: discord.gg/Mt2hD7DG

Schedule

Registration closes Wednesday, Nov 4 at 11:59 PM; you build from Thursday, Nov 5 until submissions close at 12:00 PM on Saturday, Nov 7. All times are Philippine time (UTC+8).

When	What happens
Wed, Nov 4, 11:59 PM	Registration closes (Luma RSVP and the registration form)
Thu, Nov 5	Onboarding & Kickoff on Google Meet at 9:00 PM (meet.google.com/app-dquo-nxp), then the online build starts

When	What happens
Nov 5 to Nov 7 (noon)	Build days: office hours and help on Discord
Sat, Nov 7, 12:00 PM	Submissions close
Sat, Nov 7, 4:00 PM	Top 10 teams announced
Sun, Nov 8, 1:00 PM	Face-to-face round for the top 10 teams, 1:00 PM to 8:00 PM: pitching and awarding. Certificates of participation go to every participant after the winners are announced

How to register

You are registered only when you have done both steps before Wed, Nov 4, 11:59 PM: the Luma RSVP and the registration form.

Not registered through the Google Form yet? Register now. A Luma RSVP alone is not enough. Without a submitted registration form, your project is not eligible for judging or prizes.

1. RSVP on Luma: luma.com/gjclwpkd
2. Fill out the registration form: forms.gle/wL8tPiCwGkK8YENV9

Solo or team. The form asks first. Teams have 2 to 4 members, and the team leader submits one form for the whole team, with each member's name and email.

Proof of support. The form asks for screenshots. Teams upload one screenshot per member for each item.

- ☐ Starred the Noir GitHub repo: github.com/noirx402/noir
- ☐ Starred the Velo GitHub repo: github.com/Velo-Ecosystem-Collection/Velo
- ☐ Following Noir on X: x.com/Noirx402
- ☐ Following Velo on X: x.com/RunVeloDev
- ☐ Following the community partners on X: Highrable, GigPay, TyFi

Join the Discord for updates and help during the hackathon: discord.gg/Mt2hD7DG

Build requirements

Every project must meet all six requirements to be judged.

1. **It is an agentic payment.** An AI agent decides, requests or makes the payment. A human clicking "Pay" on a normal checkout does not count.
2. **Noir: strictly through the SDK, on the deployed contracts.** Your agent must interact with the deployed Noir Testnet contracts listed in Using Noir, and only through the Noir SDK (@noirx402/agent-sdk). Deploying your own copies of the contracts, or calling other contract IDs, does not count.
3. **Velo: for the rest of your application.** Use the Velo SDK (@carts1024/velo-sdk) to build the other parts of your app, such as checkout sessions, payment status and signed webhooks.
4. **It runs on Stellar Testnet.** Never use real funds.
5. **It has a public GitHub repo** set up as a monorepo, with a README in the required format (see Repo and README format). Judges score from this README.
6. **Its website is live.** Your app must be deployed and reachable at a public URL when you

submit. A localhost-only demo does not count.

What counts as an agentic payment. The agent holds a constrained Noir agent key or acts for a user, and it chooses when, whom and how much to pay. Examples: an agent that buys API calls as it needs them, an agent that pays a merchant when a card is tapped, or an agent that settles a bill once a task is done.

Open-source code and libraries are welcome. Credit any code, libraries and ideas you use in your README.

Prerequisites

You can build this with web development skills and no prior Stellar experience. Before kickoff, make sure your team has:

- ☐ **JavaScript or TypeScript**, plus async/await
- ☐ **Node.js and a package manager** (npm, pnpm or yarn), with workspaces for the monorepo
- ☐ **Git and a GitHub account**
- ☐ **A frontend framework** for the live website (React, Next.js, Vite or similar)
- ☐ **A hosting account** for the live website and server (Vercel, Netlify, Render, Railway or similar)
- ☐ **Chrome**, for the Noir extension
- ☐ **An AI agent approach**: an LLM API, an agent framework, or rule-based logic that decides when to pay
- ☐ **The Stellar basics below**, read once

Stellar developer guide

Everything in this hackathon runs on Stellar Testnet, a free copy of the network where test XLM has no value. This is what you need to know to work with it.

Key concepts

Term	What it means here
Account / keypair	A public key (G...) is the address; the secret key (S...) signs. Never commit a secret key.
Testnet	The test network. Its passphrase is Networks.TESTNET in the SDK. Testnet can be reset, so keep setup scripts that rebuild your state.
Friendbot	Free Testnet XLM. It funds a new account with 10,000 test XLM.
XLM and stroops	XLM is the native asset. 1 XLM = 10,000,000 stroops; the Noir SDK uses stroops as bigint.
Minimum balance	An account must keep a small XLM reserve, so never drain it to zero.

Term	What it means here
Soroban	Stellar's smart contract platform. The Noir contracts are Soroban contracts (IDs start with C...).
Stellar Asset Contract	The contract form of an asset. Native XLM's is Asset.native().contractId(Networks.TESTNET).
Fees	Every transaction pays a small fee in XLM. Soroban calls cost more; a Noir card's first tap costs the agent about 2.3 XLM on Testnet.
Horizon	The API for accounts, balances, payments and transaction history.
Stellar RPC	The API for Soroban: simulating and sending contract calls, and reading contract events.
Wallet / signer	Signs transactions for the user (the Noir extension, or any Stellar Wallets Kit wallet).

Testnet endpoints

Service	URL
Horizon	https://horizon-testnet.stellar.org
Stellar RPC	https://soroban-testnet.stellar.org
Friendbot	https://friendbot.stellar.org?addr=<G... address>
Explorer	stellar.expert/explorer/testnet
Stellar Laboratory	lab.stellar.org

The Noir SDK already points at Testnet by default, so you only need these for your own calls and debugging.

First steps: create and fund a Testnet account

```
npm install @stellar/stellar-sdk
import { Keypair, Horizon } from '@stellar/stellar-sdk'

// 1. Make a keypair. Save the secret in .env, never in code.
const kp = Keypair.random()
console.log('Public:', kp.publicKey())

// 2. Fund it with Friendbot.
await fetch(`https://friendbot.stellar.org?addr=${kp.publicKey()}`)

// 3. Check the balance on Horizon.
const horizon = new Horizon.Server('https://horizon-testnet.stellar.org')
const account = await horizon.loadAccount(kp.publicKey())
console.log(account.balances)
```

You need at least three funded Testnet accounts for a Noir flow: the **card owner**, the **agent** (it

pays its own fees) and the **merchant**. The Noir extension can also create and fund a wallet for you from its popup.

Good habits

1. Keep one .env per app, with every key listed (empty) in .env.example.
2. Run agent keys only on the server, never in the browser.
3. Simulate before you send: the Noir SDK does this for you, and checkPayment tells you why a payment would fail before you pay a fee.
4. Paste every transaction hash into Stellar Expert to see what happened.
5. If a call fails with an account error, check that the account exists and has XLM for fees.

Learn more: Stellar docs · Smart contracts · x402 on Stellar · JavaScript SDK · Stellar Wallets Kit · Noir's Stellar development guide

What you need

Set these up before kickoff. Everything runs on free Testnet XLM.

Tool	What it's for	Get it
Node.js	Runs the Noir and Velo SDKs. The Noir SDK needs ^22.23.4, ^24.22.1 or >=26.12.1 (yarn refuses other versions)	nodejs.org
Noir SDK	Reads card policies, checks payments, runs authorize and claim, events and webhooks	npm install @noirx402/agent-sdk @stellar/stellar-sdk
Stellar SDK v16	Peer dependency of the Noir SDK	installed with the line above
Velo SDK	Checkout sessions, confirmed payments, signed webhooks	npm install @carts1024/velo-sdk@alpha
Velo API key	Authenticates your Velo calls on Testnet	velo-build.dev (dashboard)
Noir Chrome extension	A Stellar wallet that signs for your dApp at window.noir.stellar, links Noir cards and runs tap-to-pay	Chrome Web Store
Testnet XLM	Funds wallets, escrows and fees	Friendbot in the extension popup, or Stellar Laboratory
GitHub account	Your public project repo	github.com

Fee tip. A card's first tap costs the agent about 2.3 XLM in Soroban fees on Testnet, so fund the agent key as well as the escrow.

NFC cards are optional. The extension can link a card by typing its UID instead of scanning it, so you can test tap-to-pay without hardware.

- ☐ Node installed at a supported version (node -v)

- ☐ Noir extension installed, wallet created and funded with Friendbot
- ☐ Velo API key in a .env file, never committed
- ☐ Both SDKs install cleanly in an empty project

Using Noir

Noir links an NFC card to a constrained payment agent: the card owner picks which key may spend, which asset, how much per payment and until when. The SDK talks straight to three Soroban contracts, so there is no Noir server and no API key, and it never handles a user's secret key.

Latest Testnet contracts (redeployed 2026-10-24, built into the SDK as TESTNET):

Contract	What it does	Testnet ID
device_registry	Maps a card (SHA-256 of the NFC tag UID) to its owner's wallet	CAC4KOLHDLDYDWBOKXNKRN5KVHO56PLOIFXDLTBUWGVN6GVDEB7FQHU5
agent_registry	Holds each agent's policy: max amount, asset, expiry	CCMZDSPTLX7IIJWCYEUM5DJUHOSOYSTHGK777S2TXZWEMC5ZZGPJ3EBV
payment_escrow	Holds card funds; the agent authorizes, the merchant claims	CC7EAW5FNJEDBS7THM5IT6XIOHO4AOYMLXT3KSZ3HWUIKEACTW7E5BFS

The older 2026-10-07 IDs are retired. Projects must use the IDs above.

The payment flow, in call order:

1. The owner links the card: registerDevice (tag hash to wallet, with the agent key).
2. The owner sets the agent's policy: registerAgent (asset, maxAmount, expiresAt).
3. The owner funds the card: fundEscrow (XLM into escrow).
4. Before paying, your agent checks: checkPayment returns { ok: true } or a reason such as over_limit, expired or insufficient_escrow.
5. The agent pays: authorizePayment, which moves funds into the merchant's pending balance.
6. The merchant collects: claim.

Install and connect:

```
npm install @noirx402/agent-sdk @stellar/stellar-sdk
import { Asset, Networks } from '@stellar/stellar-sdk'
import { NoirAgentClient, keypairSigner, toStroops } from '@noirx402/agent-sdk'

const noir = new NoirAgentClient()                          // Testnet by default
const asset = Asset.native().contractId(Networks.TESTNET) // native XLM token contract

const check = await noir.checkPayment({ deviceHash, agent, asset, amount:
toStroops('2.5') })
if (!check.ok) throw new Error(`Card refused: ${check.reason}`)

const { hash } = await noir.authorizePayment({
  agent: keypairSigner(process.env.AGENT_SECRET!),
  deviceHash, merchant, asset, amount: toStroops('2.5'),
})
```

Signers. noirExtensionSigner() connects to the Noir Chrome extension, walletSigner(wallet,

address) wraps any Stellar Wallets Kit wallet, and keypairSigner(secret) is for servers and agents that hold their own key.
Amounts are in stroops (bigint, 1 XLM = 10,000,000). Use toStroops and fromStroops, never floats.
React to payments. noir.watch({ types: ['payment.authorized'], onEvent }) streams contract events, and startWebhookRelay POSTs them to your endpoint signed with HMAC-SHA256. Check each delivery with verifyWebhookSignature and dedupe on event.id.
Errors throw NoirContractError with the contract's own name, for example InsufficientBalance, AgentNotAuthorized or NothingToClaim.

Noir references

Use the SDK pages to build. The contract and architecture pages explain what the SDK is calling under the hood.
Noir website and docs: noirx402.github.io/noir

Page	What's in it
Docs home	Start page for developers
Quickstart	Get a first Noir call working
SDK	@noirx402/agent-sdk guide
Contracts	The three deployed contracts and their IDs
Extension	Connecting the Noir Chrome extension to your dApp
Webhooks	Signed event delivery to your server
API	REST gateway reference

Noir GitHub repo: github.com/noirx402/noir

File	What's in it
SDK README	Install, signers, checkPayment, authorizePayment, claim, card lifecycle, events, webhooks, networks
SDK examples	check-and-pay.ts, webhook-relay.ts, webhook-receiver.ts
Noir RPS example	A full dApp using the extension signer and a server signer
Docs index	Every developer doc, by task
API reference	SDK, REST gateway and app module
Smart contract reference	Methods, errors and the live Testnet IDs

File	What's in it
Architecture	How the parts fit, with diagrams
Walkthrough	The whole payment flow, step by step, with Testnet transactions
Stellar development guide	Stellar SDK, auth, events
Supported NFC hardware	Which NFC tags work and how they are hashed
Installation guide	Running Noir locally
Extension README	What the Chrome extension does and how dApps connect
REST gateway	For apps that can't run JavaScript
Deploy evidence	Contract IDs, WASM hashes and deploy records

Noir Chrome extension: Chrome Web Store · **npm:** @noirx402/agent-sdk · **X:** @Noirx402

Using Velo

Velo gives your agent a checkout and a source of truth: it marks a payment as paid only after the Stellar ledger confirms it, then sends your server a signed payment.succeeded webhook. The SDK is in alpha and runs server-side only.

Get a key. Sign in to the Velo dashboard at velo-build.dev with a Stellar wallet, create a project, and generate a Testnet API key. Keep it in .env as VELO_API_KEY.

Install (use the alpha tag; latest still points at an older build):

```
npm install @carts1024/velo-sdk@alpha
```

Create a checkout session:

```
import { Velo } from '@carts1024/velo-sdk'

const velo = new Velo({ apiKey: process.env.VELO_API_KEY!, environment: 'testnet' })

const { checkoutUrl, paymentIntentId } = await velo.checkout.sessions.create({
  amount: '10.00', asset: 'native', description: 'Agent purchase',
  successUrl: 'http://localhost:3000/success', cancelUrl:
'http://localhost:3000/cancel',
})

const intent = await velo.paymentIntents.retrieve(paymentIntentId)
console.log(intent.status)
```

Verify webhooks. Pass the raw, unparsed request body and the x-velo-signature header. Keep VELO_WEBHOOK_SECRET on the server.

```
const event = await Velo.webhooks.verify({
  payload: rawBody,
  signature: req.headers['x-velo-signature'],
  secret: process.env.VELO_WEBHOOK_SECRET!,
})
```

A bad or missing signature throws VeloWebhookSignatureVerificationError; return a 400 and

ignore the event.
Full reference: Velo docs · SDK README · Velo Pay demo · examples for Express and Next.js

Reference architecture and project ideas

Noir and Velo don't connect to each other; your agent connects to both. A common shape: the card owner sets limits in Noir, your agent pays within them through the Noir contracts, and your agent also uses Velo for checkout and confirmed-payment webhooks.

Part	Talks to	How
Card owner (NFC card + wallet)	Noir Testnet contracts	Registers the card, sets the agent policy, funds the escrow
Your AI agent (holds the agent key)	Noir Testnet contracts	checkPayment, then authorizePayment, through the Noir SDK
Merchant	Noir Testnet contracts	claim collects the pending XLM
Your AI agent	Velo	Creates checkout sessions; Velo confirms on the ledger
Velo	Your server	Sends a signed payment.succeeded webhook that your server verifies

The two halves only meet in your code: the Noir side limits what the agent may spend, and the Velo side gives you a checkout and a signed webhook when a payment lands. How you combine them is up to your project.

Ideas to start from:

- **Pay-per-use agent:** an AI assistant that pays for each API call or dataset it uses, within a daily cap
- **Tap-to-pay with an agent:** a shop where a card tap lets an agent check the policy and pay the merchant
- **Freelance payouts:** an agent that pays gig workers once their task is checked as done
- **Subscription manager:** an agent that renews, pauses or cancels services against a budget
- **Group expense settler:** an agent that splits a bill and pays each person's share
- **Merchant assistant:** an agent that watches payment.authorized events and claims the pending balance

Repo and README format

Every project is one public GitHub monorepo, and its root README follows the template below. Judges read this README first and score from it, so a missing section costs you points.

Monorepo layout. Keep every part of your project in one repo, with each part in its own folder:

```
your-project/
├── apps/
```

```
|   ├── web/           # the live website (frontend)
|   └── server/        # API, Velo webhooks, agent runtime
├── packages/
|   ├── agent/         # agent logic and its Noir SDK calls
|   └── shared/        # shared types and config
├── docs/              # screenshots, diagrams, extra docs
├── .env.example       # every variable, no real values
├── package.json       # workspace root
└── README.md          # the judging README
```

Use npm, pnpm or yarn workspaces (Turborepo is optional). Folder names can change to fit your project, but the root package.json must declare the workspaces and one command must install everything.

README template. Copy this into your root README.md and fill in every section, in this order:

```
# Project name

One-line pitch: what your agent does and who it is for.

**Live website:** https://...
**Demo video (2 min):** https://...
**Team:** Name (Discord handle), Name (Discord handle)

## Problem
Who has the problem and why it matters. 2 to 4 sentences.

## Solution
What you built and how the agent solves it.

## How the agentic payment works
Step by step: what the agent decides, when it pays, whom and how much.

## Architecture
A diagram (image or Mermaid) of your apps, the agent, the Noir contracts and Velo.

## Noir integration
- Noir SDK version used (@noirx402/agent-sdk)
- Which contracts you call (device_registry, agent_registry, payment_escrow)
- Which SDK methods you use, with links to the files and lines

## Velo integration
- Velo SDK version used (@carts1024/velo-sdk)
- What Velo does in your app (checkout, payment status, webhooks)
- Links to the files that call Velo

## Testnet proof
At least one Stellar Expert link to a transaction your agent made.

## Repo structure
The monorepo tree, one line per folder.

## Run it locally
1. Prerequisites (Node version, wallet, Testnet XLM)
2. `npm install` (or pnpm / yarn) at the root
3. Copy `.env.example` to `.env` and fill it in
4. The start command

## Environment variables
| Name | What it is | Where to get it |

## Screenshots
```

```
## What's next

## Credits
Libraries, code and ideas you used.
```
The judges check the Noir integration, Velo integration and Testnet proof sections against the code, so the links must point to real files.

Submission checklist

Submit before Saturday, Nov 7, 12:00 PM. The official submission link will be posted in Discord #submissions and #announcements before kickoff.

- ☐ **Public GitHub monorepo** link, with a root README in the required format
- ☐ **2-minute demo video** that shows the agent making or requesting a payment
- ☐ **Short write-up**: the problem, how your agent works, and the payment flow through Velo and the Noir contracts
- ☐ **Live website** link (required: the app must be deployed and working)
- ☐ **Team members**: names and Discord handles, all already registered through the Google Form
- ☐ **Proof on Testnet**: at least one transaction hash or Stellar Expert link from your demo

Before you submit, check that a stranger could clone your repo and run it: no secrets in the code, a .env.example file, and setup steps in order.

Judging and pitching

Projects are scored on five criteria. The top 10 are announced Saturday, Nov 7, at 4:00 PM and pitch face-to-face at 1:00 PM on Sunday, Nov 8.

Criterion	Weight	What judges look for
Agentic payments fit	30%	The agent truly decides, requests or makes the payment
Technical execution	25%	Velo and the Noir contracts work end to end on Testnet
Innovation	20%	A new idea or a new use of agent payments
Real-world potential	15%	Someone would use it beyond the hackathon
Pitch and demo	10%	Clear story, working live demo

Pitch format: a 5-minute pitch with a live demo, then 3 minutes of Q&A. Bring a laptop with the demo running and a backup video in case the network fails.

Rules and code of conduct

Breaking these rules can get your messages removed, you muted or banned, or your project

disqualified, at the organizers' discretion.

1. **Be respectful.** No harassment, hate speech, discrimination or personal attacks.
2. **No scams or shilling.** No token promotions, airdrops or unsolicited DMs. The Noir team will never DM you first asking for funds.
3. **Protect your keys.** Never share a seed phrase, secret key or API key, and never commit one to GitHub. No one from the team will ever ask for them.
4. **Testnet only.** Projects run on Stellar Testnet. Don't send real funds while testing.
5. **Give credit** for code, libraries and ideas you use.
6. **Stay on topic** in Discord: questions in #dev-help, Noir talk in #noir-sdk-and-extension, Velo talk in #velo-integration, demos in #showcase.
7. **English or Filipino**, so everyone can follow along.
8. Follow Discord's Terms of Service and Community Guidelines.

Getting help

Join the Discord for updates and help during the hackathon: discord.gg/Mt2hD7DG. The Noir and Velo teams answer in the channels below during the build.

Channel	Use it for
#hackathon-info	Schedule, rules and official updates
#find-a-team	Post an intro to find teammates
#hackathon-help	Questions about the event
#dev-help	Build questions and bugs in your code
#noir-sdk-and-extension	Noir SDK, contracts and Chrome extension
#velo-integration	Velo SDK, checkout and webhooks
#submissions	Submission link and deadlines
Office Hours (voice)	Talk a problem through live

Found a bug in Noir itself? Report it: forms.gle/MrfvC8NaFFFogcRt8

Links

What	Link
Luma RSVP	luma.com/gjclwpkd
Registration form	forms.gle/wL8tPiCwGkK8YENV9
Noir GitHub	github.com/noirx402/noir
Noir SDK	@noirx402/agent-sdk
Noir Chrome extension	Chrome Web Store
Velo GitHub	github.com/Velo-Ecosystem-Collection/Velo

What	Link
Velo docs	velo-build.dev/docs
Stellar docs	developers.stellar.org
x402 on Stellar	developers.stellar.org/docs/build/agentic-payments/x402
Testnet explorer	stellar.expert/explorer/testnet
Noir on X	x.com/Noirx402
Velo on X	x.com/RunVeloDev

Community partners: Highrable, GigPay and TyFi.